Boston University

OpenBU **http://open.bu.edu**

Boston University Theses & Dissertations Boston University Theses & Dissertations

2016

Identifying unsoundness of call graphs in android static analysis tools

https://hdl.handle.net/2144/17085

"Downloaded from OpenBU. Boston University's institutional repository."

BOSTON UNIVERSITY

COLLEGE OF ENGINEERING

Thesis

IDENTIFYING UNSOUNDNESS OF CALL GRAPHS IN ANDROID STATIC ANALYSIS TOOLS

by

DANA ALJAWDER

B.S., American University of Sharjah, 2011

Submitted in partial fulfillment of the

requirements for the degree of

Master of Science

2016

© 2016 by
DANA ALJAWDER
All rights reserved

Approved by

First Reader

Manuel Egele, Ph.D.
Assistant Professor of Electrical and Computer Engineering

Second Reader

Ayse Coskun, Ph.D.
Associate Professor of Electrical and Computer Engineering

Third Reader

Douglas Densmore, Ph.D.
Associate Professor of Electrical and Computer Engineering

Acknowledgments

First and foremost, I would like to begin by thanking my advisor Professor Manuel Egele for a number of things including an unlimited amount of advice, support and patience. I truly appreciate the time spent working on this project and even more the help and guidance from my advisor. The experiences I have gained by working on this thesis are priceless and I attribute them to Professor Egele.

I would like to thank my committee members, Professor Ayse Coskun and Professor Douglas Densmore, for their time and valuable feedback.

A very special thank you to my parents, who happen to be my mentors in life, for their encouragement and support. Without their encouragement, I would not have been able to push through difficult times. I should also thank my parents for my four siblings, and my siblings themselves, who helped in shaping the person that I am today.

Finally, I would like to thank my local support team, Bishwa Pandey, Lama Alsuwayan, and Noor Turaif, for the coffee, food and unconditional positivity.

Dana Aljawder

Master's Student

ECE Department

IDENTIFYING UNSOUNDNESS OF CALL GRAPHS IN ANDROID STATIC ANALYSIS TOOLS

DANA ALJAWDER

ABSTRACT

Analysis techniques are used to test mobile applications for bugs or malicious activity. Market operators such as Google and Amazon use analysis tools to scan applications before deployment. Creating a call graph is a crucial step in many of the static analysis tools for Android applications. Each edge in a call graph is a method call in the application. A sound call graph is one that contains all method calls of an application. The soundness of the call graph is critical for accurate analysis. Unsoundness in the call graph would render analysis of the application flawed. Therefore, any conclusions drawn from an unsound call graph could be invalid.

In this project, we analyze the soundness of static call graphs. We propose and develop a novel approach to automatically identify unsoundness. We create a dynamic call graph to examine the soundness of the static call graph. We map the edges of the two graphs. Any edge observed dynamically but not present in the static call graph is a witness for unsoundness. We show that there are edges in the dynamic call graph that are not contained in the static call graph. We analyze 92 applications to find a total of 19,653 edges missed by a state-of-the-art static analysis tool. To further analyze these edges, our tool categorizes them into groups that can help identify the type of method call that was missed by the static analysis tool. These categories pinpoint where further research efforts are necessary to improve current state-of-the-art static analysis capabilities.

Contents

1 Introduction **1**

2 Background & Motivation **4**

2.1 Background 4

2.1.1 Android Environment 4

2.1.2 Android Applications 5

2.1.3 Android Application Security 9

2.1.4 Program Analysis 9

2.2 Motivation 13

3 Approach **14**

3.1 Static Call Graph 14

3.2 Dynamic Call Graph 15

3.3 Mapping 16

4 Implementation **18**

4.1 Static Analysis 18

4.2 Dynamic Analysis 19

4.3 Mapping the Call Graphs 22

4.4 Categorization 24

5 Evaluation **29**

6 Related Work **40**

7 Conclusions **43**

References **44**

Curriculum Vitae **47**

List of Tables

5.1 App Statistics that show the number of applications with specific categories of edges missed 35

5.2 The numbers of missed edges in the categories 35

List of Figures

2·1 Diagram of Android Architecture EazyTutz (2015) 6

2·2 Lifecycle of an activity Android (2016) 8

3·1 System Overview 15

4·1 Example of method signature in FlowDroid's call graph 19

4·2 Code addition to `RunAnalysis` in Infoflow.java 19

4·3 Example of bytecode method signature 22

4·4 Code addition to Dalvik in AOSP where `contextDir` is the preliminary categorization and 10045 is specific to the emulator instance we used. 22

4·5 Hierarchy of an Application Package 26

4·6 Interaction between Application and Framework 27

5·1 `OnClickListener` in the XML 30

5·2 `OnClickListener` in the Java Code 30

5·3 Code from the second app, divide by zero exception 31

5·4 Code from the third app, extension of Toast class 32

5·5 Code from the fourth app, extension of Exception class 33

5·6 Distribution of missed edges per app 36

5·7 Missed edge from an application 37

5·8 The code 38

Chapter 1

Introduction

We rely heavily on mobile applications to enhance our daily lives. Smart devices and mobile applications are increasingly becoming more the tool of today's connectivity with the external world for leisure, social and business transactions. They legitimately store, manage and transmit vast amounts of sensitive personal data. Sensitive data can range from a phone's unique identifier, personal information, geographical locations, emails, banking transactions, and government transactions. Klieber et al. (2014); McClurg et al. (2013).

Mobile applications are offered through App Stores. Uploading mobile applications to App Stores is as easy as creating an account. This happens to be the minimum requirement to submit an application to the official Android market (Google Play) Android (2016). According to Ben EvansEvans (2015), the number of applications downloaded from Google Play increased from 32 million to 50 million from 2013 to 2015. More than 80% of mobile applications in Google Play are offered free of charge which could explain the rise in the number of applications downloaded.

The enhancements that mobile applications provide us come at a price. In May, 2015, the MIT Technology Review wrote about the perils of Google Play's "lighter touch in vetting apps" Review (2015). The fact that devices and applications are storing, managing and transmitting such sensitive data and information made the devices targets for attacks by malicious applications. These attacks have drawn attention and action from industry and research communities to deal with the issue of

data leaks, that is, sensitive data transmitted by the device over the Internet or via SMS to unintended destinations and parties or stored in publicly accessible storage Arzt et al. (2014); Backes et al. (2015); Chin et al. (2011); Li et al. (2014).

Large application development companies and App Stores like Google Play and Amazon use analysis techniques to test mobile applications for bugs or malicious activity before deployment. There are two classes of application analysis: static and dynamic. Static analysis is performed on an application when it is not running and dynamic analysis while it is running. Both techniques are used to find vulnerabilities or malicious activities. An analysis tool is considered sound when the results produce no false negatives. The tool is considered complete when the results produce no false positives. For example, a tool that detects information leaks is sound when no leaks are missed and complete when it does not falsely report a leak.

Ideally an analysis tool would be sound and complete to ensure the full detection of data leaks or malicious behaviors. Rice's Theorem Rice (1953) states that for any non-trivial program property, such as soundness or completeness, no general automated method can determine whether or not this property holds. This means that a sound and complete tool cannot be automated. Previous work extends or improves existing static analysis tools so that it can be more sound. In this paper we look to identify unsoundness in existing static analysis tools. Reducing the number of false negatives produced from an analysis reduces the risk of a malicious or vulnerable application passing as benign. We aim to pinpoint weaknesses in the tools themselves.

A call graph, $CG = (V,E)$, is a graph that represents method calls of an application. Each vertex, V, in the graph is a method and each edge, E, is a method call. A sound call graph is one that contains all edges of an application. Unsoundness in the call graph would render analysis of the application flawed and any conclusions drawn from the call graph could be invalid. For example, if an analysis tool unknowingly

uses an unsound call graph to identify whether a certain series of method calls exists in a program, and an edge within the series is missed due to unsoundness in the call graph, the analysis will incorrectly report that the series does not exist in the program. In this project, we use dynamic analysis to identify edges that are missed in the call graphs in static analysis. Any edge found dynamically which is not found in the static call graph will be proof of unsoundness. We developed a tool, `EdgeAnalyzer`, that compares a call graph created by a static analysis tool to a call graph created dynamically to find and categorize missed edges.

We modified a static analysis tool to extract its call graph, used dynamic analysis to extract a dynamic call graph, and used `EdgeAnalyzer` on a group of real world applications. We show that there are a number of categories of missed edges from the static call graph. `EdgeAnalyzer` automatically identifies the areas a static tool will need to improve on to increase the soundness of its analysis.

The contributions we make in this thesis are as follows:

1. Develop and evaluate a system that automatically identifies and categorizes edges missing from the call graph in static analysis tools.

2. Use a state of the art static analysis tool to demonstrate the effectiveness of our system.

3. Analyze the results to examine causes for the identified unsoundness of call graphs.

Chapter 2

Background & Motivation

2.1 Background

In this chapter we discuss the Android platform, analysis techniques and their related issues.

2.1.1 Android Environment

Prior to delving into the specifics of how Android applications function and the complexity that could result in malicious or unintended activity occurrences, the following is a brief description of the Android environment and its framework.

Android is an open source, mobile operating system based on Linux. It is a software stack of the following five layered components (refer to the Android architecture diagram in Figure 2·1) Enck et al. (2014); Sharma (2011).

1. Kernel. At the base of the software stack is the Kernel which acts as an abstraction layer of the hardware providing a standard interface between the hardware and the Android software stack.

2. Android Runtime. This layer includes the Dalvik Virtual Machine, which has been replaced by Android Runtime (ART) since version 5.0. Both ART and Dalvik compile the bytecode of an application into native code. Dalvik uses a just-in-time (JIT) compiler where ART uses an ahead-of-time (AOT) compiler. In this project we focus on the Dalvik Virtual Machine where the Dalvik

bytecode is compiled at runtime.

3. Libraries. The layer consists of Android native libraries written in C/C++, including Surface Manager to manage the user interface, Media Framework to support various media formats, System C Libraries, and SQLite.

4. Application Framework. This layer offers developers APIs to allow for interaction with the libraries in the lower layer. Included in these APIs are the Activity Manager which manages various stages of the application life cycle, the Notification Manager to enable the creation of customized alerts, the View System to create layouts and many more.

5. Applications. The top layer of these components is the application layer. This layer manages the applications running on a device to allow them to communicate with the application framework layer.

2.1.2 Android Applications

Application Components

Android applications consist of any of four components which are the basic units of an Android application; activities, services, content providers, and broadcast receivers. An activity is a graphical user interface (GUI) of an application. It provides the user with an interface to enable interaction, for example requesting a list of the contacts available on the device. Each activity is a separate interface. An application usually has more than one activity. A service is a task that runs in the background in parallel to other application tasks. An example of a service is a music application playing music in the background while the user is browsing the Internet. A content provider is a database for internal data storage and is a standard interface for data exchange between inter-application and intra-application components. A broadcast receiver

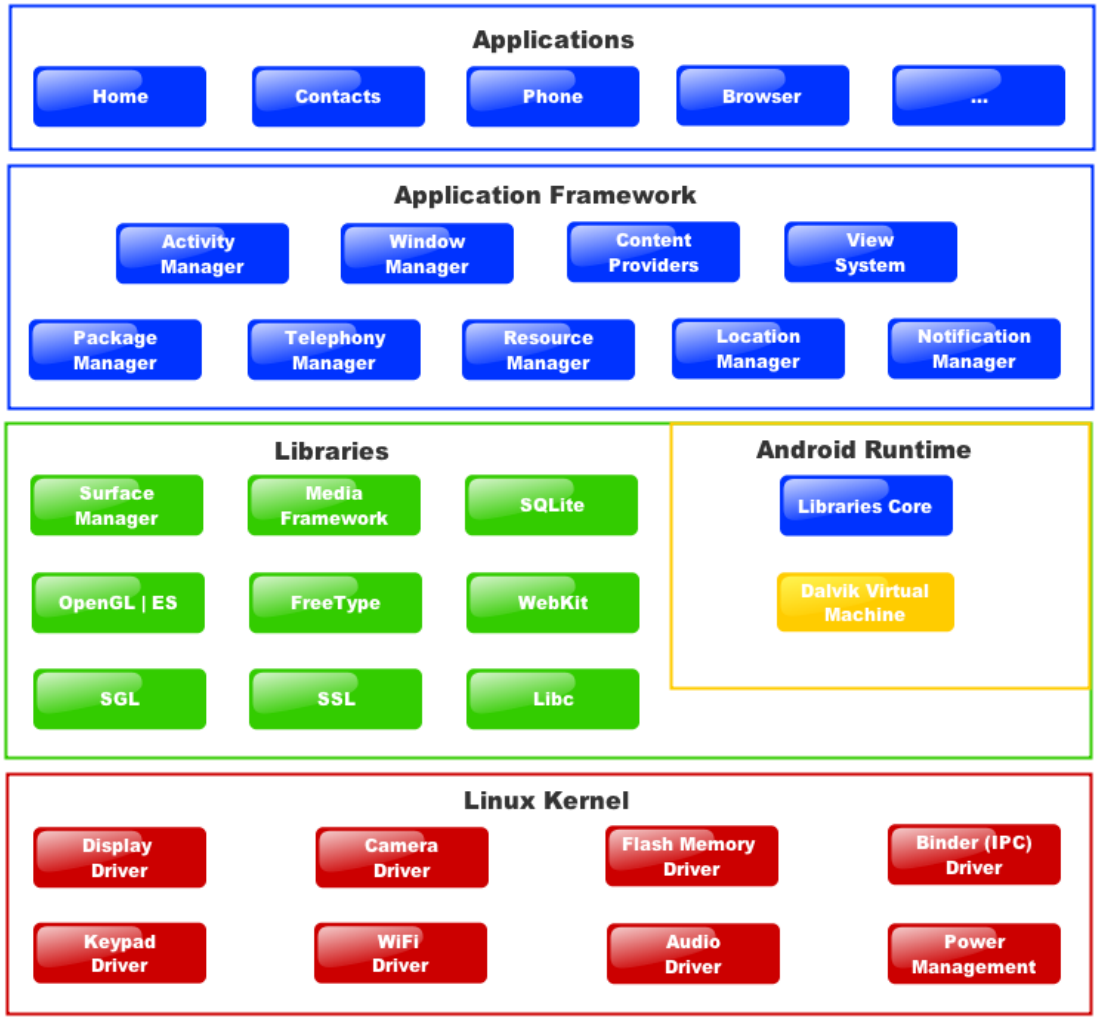

Figure 2·1: Diagram of Android Architecture EazyTutz (2015)

receives and reacts to broadcast messages from other components or the system. The broadcast messages allow developers to trigger events based on the message received. For example, if the music application that is playing music in the background receives a message from the device about an incoming phone call, the developers can use this event to pause the music until the application receives a message that the call has ended. Developers describe these four components of their applications in the manifest file of the application (i.e., `AndroidManifest.xml`).

Entry Points

Each of the four components has several entry points. An android application does not have one standard start method. In other words, the start of an execution of an application differs. Each entry point can serve as a starting point of an execution. Figure 2·2 shows the lifecycle of an activity. The square rectangles in the diagram represent the entry point of an activity component. Android applications are event driven which makes reasoning about the possible order of execution difficult.

Android APK file

Android Application Package(APK) is the file format used to distribute and install applications onto the Android operating system. The APK file is a compressed zip file that contains the compiled program classes, resources and assets, certificates and the manifest file Android (2016).

Application Manifest File

Android application's manifest file is the configuration file of an application Chin et al. (2011). Since the file declares the application's four components as mentioned earlier, it is important to note that the interactions of components and listeners can be retrieved from this file Android (2016). Additionally, the manifest file contains

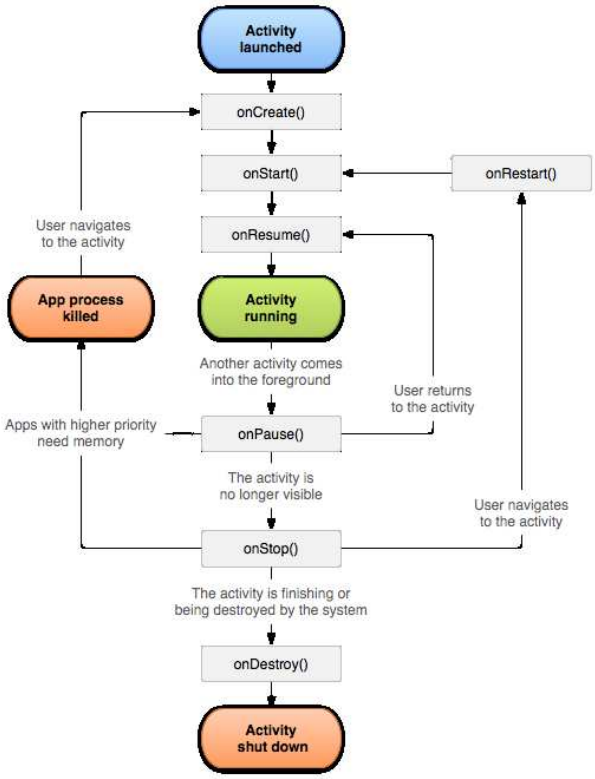

Figure 2·2: Lifecycle of an activity Android (2016)

permissions granted to the application. For an application to be able to use system resources, such as the device's camera, the permissions need to be set and the user must accept the permissions on installation.

Application Layout files

The application layout files define the user interface of the application. They contain the definition of the format of the screen's layout. Elements in the layout, such as buttons, can have their properties defined in the layout file. Properties of a layout element can include size, position, color, and a method for a click action. The user interface layout could be declared in one of two ways; using XML or be instantiated during runtime. For example, the method to be called when a button is clicked can be defined in either the XML file or the Java code.

2.1.3 Android Application Security

Each Android application on a device is granted a user ID (UID) upon installation. The UID is used to set file permissions to the application the way that Linux does. Therefore if the UID is granted read/write permissions to folder "foo", then the application has these permissions. The permissions granted to the application in the `AndroidManifest.xml` file allow the application to interact with system components. For example, if an application requires access to the device's camera, the permission to access the camera is set in the manifest file. These permissions must be accepted by the user on installation, otherwise the application will not be installed.

2.1.4 Program Analysis

In program analysis there are two well-known classes of techniques researchers and developers use to detect and protect against vulnerabilities and malicious activities. They are static analysis and dynamic analysis.

Dynamic Analysis

Dynamic analysis is used to detect applications' vulnerabilities and data leaks at runtime by executing and monitoring applications' behaviors. Dynamic analysis tools can monitor control flows and data flows of sensitive data and critical services such as connection to remote servers or sending SMSs and report any detected vulnerabilities Spreitzenbarth et al. (2015). As such, analysis of applications requires contextual information about user input, data sources and destinations. Not all parts of an application will be executed in the case of dynamic analysis since contextual information is limited by the specific instance of the execution Enck et al. (2014); Li et al. (2014). Since the analysis occurs at runtime, the details of the run are precise. Any statement made about the program based on the analysis will only be valid for the specific run. Obfuscated code is code that is deliberately written in a way that is not human understandable. Obfuscation techniques can hide system activities by calling functions outside the Dalvik/Java runtime library Spreitzenbarth et al. (2015). In contrast to static analysis, in the case of obfuscation techniques, dynamic analysis handles these techniques well, but a weakness of dynamic analysis is that it can be detected and avoided by malware applications as they execute Octeau et al. (2013).

Static Analysis

The term static analysis is used to describe a mechanism of analyzing applications that are not running. A static analysis tool takes the code of an application, analyzes the content of the code based on the criteria developed for the tool Chess and McGraw (2004). For example, data-flow based analysis will attempt to map all the possible data flows of an application. Static analysis can be used to make statements about entire programs. In other words, the statements made about a program are valid for all executions of the program. For example, if static analysis is used to detect

memory leaks Xie and Aiken (2005), the tool should be able to report that a program has no memory leaks.

Call graph

Many static analysis tools use call graphs as a crucial element in analyzing applications. A call graph, *CG=(V,E)*, is a directed graph with methods as vertices *V* and method calls as edges *E*. The edge's vertices are the current method of the application, caller, as the source and the method to be called, callee, as the target. Call graphs map the paths of the methods called in a program. A static call graph maps the possible method calls. This is meant to ensure the soundness of a graph, that is, all possible function calls in the application will be mapped in the call graph. As a result some edges included in the static call graph may not actually exist in the application itself Chugunov et al. (2002).

Soundness

The output of static analysis tools can produce false positives and false negatives. False positives are occurrences where the tool falsely flags an application as vulnerable or malicious. False negatives are occurrences where the tool falsely flags an application as secure. A false negative is dangerous since it gives a false sense of security. Static analysis is said to be sound with respect to a certain property if there are no false negatives. In the context of call graphs and this thesis, we specify the property to be *The method call is represented in the call graph.* In other words, a call graph is sound when every method call is represented by an edge in the call graph. A trivial solution that would render a call graph sound is creating a fully connected graph. That is a graph where all vertices are mutually connected. The fully connected graph will guarantee that all method calls are represented in the call graph. The graph will also contain method calls that do not exist in the application (false positives). A large

number of false positives will render the call graph useless. Sifting through the graph to manually verify the edge's existence will be the only way to derive meaning from a fully connected graph.

Completeness

An analysis is considered complete when it contains no false positives. The results of a complete analysis are always true but do not necessarily reflect the entire application. In the context of call graphs with respect to our property, a call graph is complete when every edge in the call graph represents a valid method call in the application. In other words, there are no edges in the call graph that represent a method call that does not exist in the application. A trivial solution to render a complete graph is by creating a disconnected graph, that is a graph with no edges connecting its vertices. The disconnected graph will guarantee no false edges in the call graph. The disconnected graph is useless as it does not give any information about the application.

Soundiness

In this thesis, we focus on soundness of call graphs so it is necessary to understand the difficulties in producing sound analysis that does not render the analysis useless. As mentioned earlier in this Chapter, we say an analysis is sound if there are no false negatives and an analysis is precise (or complete) if there are no false positives. "In Defense of Soundiness: A Manifesto" discusses the idea that the results of static analysis tools cannot be made sound without compromising scalability and/or precision Livshits et al. (2015). The authors believe that soundness in static analysis is unachievable. Their "Manifesto" advocates "Soundy" analysis, that is, analysis that attempts to "do the right thing". In their view the right thing is to be as sound as possible while not sacrificing scalability and/or precision and achieving the balance between over-approximation and intentional under-approximation. They also

see the need for some efforts to compare the results of an unsound analysis (static) and dynamic analysis results.

2.2 Motivation

We know that the complexity of the Android Framework contributes to the difficulty in making static analysis sound. Many static analysis tools use a call graph as the crucial component of the analysis of Android applications Arzt et al. (2014); Barros et al. (2015); Chin et al. (2011); Li et al. (2014). For a call graph to be sound, all edges in the application must be contained in this call graph. An unsound call graph will render any assumptions made based on its soundness invalid.

Therefore we wanted to develop a tool that can automatically find edges missed in a static call graph and then classify these missed edges. The idea is to find classes of edges that are missed in creating the static call graphs. In other words, we would like to show that static call graphs have various classes of edges missing and provide a tool that can automatically find and classify edges missed in the creation of the call graph in static analysis tools. We want this tool to pinpoint possible areas for improving soundness of static analysis tools.

Chapter 3

Approach

As explained in the previous chapter, the goal of this project is to identify the missing edges in a static call graph. We use a static analysis tool to extract the static call graph and a modified version of the AOSP (Android Open Source Project) to dynamically trace all function calls within an application. We then map the edges found during static analysis to those found during dynamic analysis. All the edges that belong to the dynamic call graph that are not contained in the static call graph are then categorized. To automate the mapping and categorization, we developed a system that combines these steps to give a final output with the statistics and details of the categories of the missed edges. Figure 3·1 shows an overview of how our system works. In the following sections we describe each of the steps mentioned.

3.1 Static Call Graph

Static analysis of Android applications attempts to capture a program behavior by analyzing its code. To create a call graph, the analysis tool must map out all possible method calls. As stated in Chapter 2.1.4, the call graph is sound if every method call is represented in the call graph. Based on the assumption that the static call graph is sound, it should contain all the possible function calls in the application. If the call graph does not, then it is unsound. To determine the unsoundness in the call graphs created by static analysis tools, we need a way to extract the static call graph. The idea behind this step is to modify the static analysis tool such that we can save the

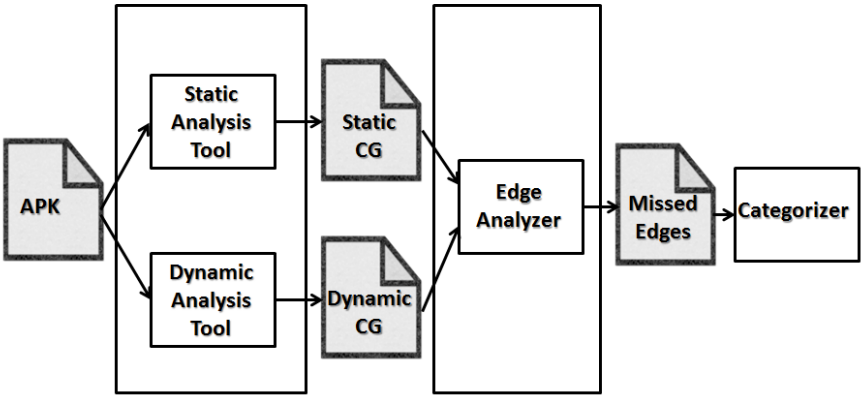

Figure 3·1: System Overview

call graph created in order to use it in later stages of the analysis.

3.2 Dynamic Call Graph

We focus on the soundness of the call graph created by the static analysis tool. To assess the soundness of the static call graph, we need to create a dynamic call graph and compare the two graphs. Dynamic analysis of Android applications cannot provide sound results as the analysis will only be performed on the specific run. In other words, not all method calls in the application will be in the call graph. Despite the fact that a vulnerability or malicious activity caught by dynamic analysis is correct, a complete set of vulnerabilities/malicious activities is not guaranteed. The results of the analysis will be based on the specific instance of the execution.

To create the dynamic call graph, we install an Android application on an emu-

lator that runs a modified version of AOSP. This version logs method calls for user installed applications. When an application is run on a framework such as Android, the system differentiates between its own factory installed applications and user installed applications using a UID (Refer to Chapter 2.1.3). We use the UID to identify the methods that belong to the installed application. We need to be able to make the distinction of whether an application is factory installed or user installed to be able to only log the methods of the user installed applications. The distinction is important because before a user installed application is started, the mobile phone's operating system uses factory installed applications to simulate actions such as opening the phone's menu or selecting an application to start. Logging method calls that we do not require, such as method calls of factory installed applications, would take extra space.

The logging functionality should output a call graph. The modified version of AOSP logs every method call made during the execution of an application to a file. To create a call graph, the file includes the current method and the method to be called (caller and callee respectively). Since this dynamic call graph will be based on a specific instance of execution, we cannot guarantee that all methods will be called but we will know that the information logged is factual.

3.3 Mapping

We state that any edge observed dynamically but not present in the static call graph is a witness for unsoundness. We define the sets as follows:

$$CG_s = (V_s, E_s)$$

$$CG_d = (V_d, E_d)$$

$$W = \{e \in E_d \mid e \notin E_s\}$$

where CG_s is the static call graph, V_s is the set of vertices of the static call graph, E_s is the set of edges of the static call graph, CG_d is the dynamic call graph, V_d is the set of vertices of the dynamic call graph, E_d is the set of edges of the static dynamic graph, and W is the lower bound on the set of witnesses of unsoundness. Based on the assumption that the static call graph is sound, all the edges found dynamically should belong to the set of edges found statically, and similarly for all vertices. In other words:

$$V_d \subseteq V_s$$

$$E_d \subseteq E_s$$

which would imply that

$$W = \emptyset$$

If we find that W is not empty, then the static call graph is unsound.

To be able to map the static call graph to the dynamic call graph, we require both call graphs. By traversing the dynamically created call graph, comparing edges to their equivalent edges in the static call graph, we can ensure every edge is contained in the static graph. Once an edge in the dynamic call graph is not found in the static call graph we log this edge to a list. At the end of this mapping phase, we will have a list that contains the edges found dynamically but missed statically.

Chapter 4

Implementation

Our tool, `EdgeAnalyzer`, uses a call graph created in static analysis tools and a call graph created by using dynamic analysis to compare and find edges missed statically. In both call graphs, each method is a vertex in the graph and the vertices are connected to create the edges. The edges represent method calls.

4.1 Static Analysis

`EdgeAnalyzer` was developed independent of any static analysis tool. `EdgeAnalyzer` can accept input from any static analysis tool in the form of a call graph. We decided to use FlowDroid as our static analysis tool as it is a state of the art precise context, flow, field, object-sensitive and lifecycle-aware static analysis tool for Android applications Arzt et al. (2014). One of the advantages of using FlowDroid is that it is an open-source tool. To be able to analyze the data from FlowDroid's call graph, we modified its source code to export the call graph to a file. The file follows the format of a dot file GraphViz (2016). The format of the edges in the FlowDroid call graph follows the Java method signature formatting. Figure 4·1 shows an example of a method signature in FlowDroid's call graph. The method `dispatchActivityCreated` belongs to the class `Application` and takes as input two parameters of type `android.app.Activity` and `android.os.Bundle`. The `Application` class belongs to the `android.app` package.

FlowDroid takes an apk file as input. The apk file is unzipped and the manifest,

```
android.app.Application: void
dispatchActivityCreated(android.app.Activity,android.os.Bundle)
```

Figure 4·1: Example of method signature in FlowDroid's call graph

```
List<SootMethod> ePoints = Scene.v().getEntryPoints();
visit(Scene.v().getCallGraph(), ePoints.get(0));
dot.plot("soot_callGraph"+ dot.DOT_EXTENSION);
```

Figure 4·2: Code addition to `RunAnalysis` in Infoflow.java

dex, and layout files are parsed. Flowdroid then uses the three files to create a call graph using a modified version of a tool named soot Vallée-Rai et al. (1999). To extract the call graph, we modified the `RunAnalysis` method in the `Infoflow` class of FlowDroid. The method `RunAnalysis` builds the call graph, initializes the flow analysis, finds the sources and sinks in the application and finds flows. A source in this case is an entry point to a component, and a sink is a point where data can be leaked. We added a method named `visit` that traverses the call graph created by soot. FlowDroid recursively finds the entry points of an application and adds them to the call graph. `visit` starts at any entry point of the application. The call graph should contain all entry points and therefore we do not need to identify all the entry points, one is enough to start the process. As it traverses the call graph, `visit` writes every edge into the dot file. We modify `RunAnalysis` at the point after the call graph is constructed and before the flow analysis begins (See Figure 4·2 for the added code). Flow analysis is expensive and not required for the call graph Arzt et al. (2014). Thus, we can omit this expensive analysis for our purposes.

4.2 Dynamic Analysis

We used version 4.3_r1 of the AOSP to build our own version that creates a dynamic call graph. We modified the code to log every function call invoked by a running user

application to a log file. The same code we modified can be used for any version of the Android source code below 5.0. The reason this code will not work for Android version 5.0+ is that Android abandoned the Dalvik architecture and moved to ART (See Chapter 2.1.1).

There are three execution modes for the Dalvik VM; fast, portable and JIT(Just In Time) Android (2016). Portable is implemented purely in `C++` and is not platform specific so is expected to run on multiple platforms. On the other hand, fast and JIT incorporate assembly and are platform specific. The semantics of all three are the same. We opted to modify the `C++` files related to the portable execution of the Dalvik VM. When a method is invoked in an application, the Android framework calls a series of `C++` methods to process and set the parameters. There are five types of method calls: `invoke-static`, `invoke-virtual`, `invoke-super`, `invoke-direct`, and `invoke-interface`. `invoke-static` is used to call a class method without a reference to a class object. `invoke-virtual` is used to call a class method with reference to an object of the class type. In other words, `invoke-virtual` requires an instance of an object to be called. `invoke-direct` invokes any of static, private, or constructor without an instance of the class object. A private method is a method that can only be accessed by methods in the same class. `invoke-interface` requires an instance of an object and is used when the reference is of an interface type. An interface type method call is one who's parent method is an abstract method. `invoke-super` is used when calling methods using `super`. `super` is used to call a method from the parent class of the current class. Each of these method types has its own opcode and format. We need to be able to log all methods regardless of method call type.

In the source code of the Dalvik portable, all five method call types are processed by the `invokeMethod` method. We modified this particular method to first check whether we are inside a user installed application. We use the `getuid()` method to

retrieve the integer that represents the application ID granted to the application upon installation. The UIDs for user installed applications are within a known range. We manually checked the UID of the first user installed application on our emulator and the number was 10046. To ensure we log the methods of user installed applications we check that the UID is greater than 10045 (see Figure 4·2). Using `invokeMethod`, we capture the information of the function call that is about to be made. This information is used to create the dynamic call graph. We achieve this by making the source of our dynamic call graph's edge the current method that is being processed (`curMethod`), and the target the method to be called (`methodToCall`); each of these variables is of type `Method`. The information extracted from these variables includes the full signature of each method (See Figure 4·2 for a snippet of the added code). The full signature of a method consists of the method name, the class it belongs to, the return variable type and the parameter types (see Figure 4·1 for an example). Additionally, the type of the call about to be made is analyzed and tagged with a category. There are two elements that interact via method calls to render an application and its functionality; the application code and the framework code. These two elements interact within themselves and with each other. Therefore, we divided the calls into four categories: within the framework, within the application, from the framework to the application, and from the application to the framework. We discuss these categories further in the categorization section of this chapter (Chapter 4.4). We write the source method and the target method as an edge with their categorization tag to the log file.

Figure 4·3 shows the format of a method signature in bytecode. The method `dispatchActivityCreated` is the same method shown in Figure 4·1. The format of the edges logged in this stage makes use of bytecode method signature. In Android, primitive types directly contain values and non-primitive types are references to ob-

```
Landroid/app/Application; dispatchActivityCreated
(Landroid/app/Activity;Landroid/os/Bundle;)V
```

Figure 4·3: Example of bytecode method signature

```
if ((int)getuid() > 10045)
{
   char *methodToCallDesc =
   dexProtoCopyMethodDescriptor(&methodToCall->prototype);
   char *curMethodDesc =
   dexProtoCopyMethodDescriptor(&curMethod->prototype);

   fprintf(self->logfile,"%c %s %s %s -> %s %s %s \n",contextDir,
      curMethod->clazz->descriptor,curMethod->name,curMethodDesc,
      methodToCall->clazz->descriptor,methodToCall->name,
      methodToCallDesc);
}
```

Figure 4·4: Code addition to Dalvik in AOSP where `contextDir` is the preliminary categorization and 10045 is specific to the emulator instance we used.

jects. Primitive types in Android are represented by a single character. Non-primitive types are represented by the character `L` Android (2016) and have their class names in a variable in the `C++` code. We retrieve the class names from the variables when required.

4.3 Mapping the Call Graphs

The dot file and the log file, are then parsed and each edge from the log file has its vertex's format modified to FlowDroid's method signature (see Figure 4·1). The tool takes 4 inputs, the log file, the dot file, the application's package name, and a number we will call traceback. The traceback number is used to back track through the dynamic graph when a missed edge is found. The traceback number represents the number of edges leading to a missed edge to be recorded. The trace of edges that

lead to the missed edge is logged to a file. We use this information to manually trace individual missed edges to help in their analysis.

The method signatures from the log file need to have their format altered to match the method representation from FlowDroid which is the same as Java's representation. The log file extracted from the emulator has its vertices (methods) formatted following Android's method and class representation. We chose FlowDroid's instead of Android's formatting because it is human-readable.

We used the `org.jgrapht` package to manage the graphs in Java. Each graph object, the static and dynamic, is created by reading the file, formatting the vertex if required, and adding the vertices and edges to the respective graph object.

Once both call graph objects have been populated, the tool traverses the dynamic call graph and compares its edges to the static one. We do this by initially trying to find the source vertex of the dynamic graph in the static graph using `org.jgrapht`'s `containsVertex` method (See Algorithm 1).

Algorithm 1 Find Missed Edges

```
procedure MapGraphs
    for each vertex v ∈ dynamicGraph do
        if staticGraph.containsVertex(v) then
            for each edge e ∈ dynamicGraph.incomingEdges(v) do
                if dynamicGraph.getEdge(e) = NULL then
                    MissingEdges ←e.
        else
            for each edge e ∈ dynamicGraph.incomingEdges(v) do
                MissingEdges ←e.
```

At the end of this process we will have a Java List of edges that are in the dynamic graph but missing from the static graph. In creating the dynamic call graph, we labeled the edges with one of the four main categories. The tool filters out the edges that are contained within the Android framework. However, we do take

into consideration edges with a source as a method from the application and target as a method from the framework and vice versa. An edge with a source as a method from the framework and target as a method from the application could potentially be a callback. A callback is any method that is called by another method which takes the first method as a parameter. In this case the first method would be a method in the application and the second a method from the framework.

When a missed edge is going to be added to the list, we use the traceback number, the dynamic graph and the source vertex to create a log of traced back edges that led up to this missing edge. We traverse the graph backward starting at the source vertex and record each edge. Once the traversal is complete, we log the missed edge with its incoming path. When the tool has completed, this file will contain all the missed edges and their log of traced back edges.

4.4 Categorization

Once the entire dynamic call graph has been traversed, the results of the saved missed edges are analyzed. As previously mentioned, there are two elements that interact via method calls in an application; the application code and the framework code. Based on these two elements we create the following four categories:

1. Edges that are within the application itself
2. Edges that are within the framework
3. Edges that originate from the application and terminate in the framework
4. Edges that originate from the framework and terminate in the application

Edges within the application represent calls of methods that are found in the application's APK file, that is, the source and target vertices are from methods found in

the APK file. The methods that could be found in the APK file include methods developed for this application and 3rd party libraries imported into this application. Android applications are developed in Java. In other words, the framework is divided into two parts; the Java framework and the Android framework, where the Android framework handles Android specific features such as the components. Edges within the framework represent interactions of methods that belong to the Android framework or the Java framework, that is, the source and target vertices are from methods that belong to either the Android framework or the Java framework. An example of an intra-framework method call would be a method from the Java `ThreadLocal` class calling a method from the Java `Thread` class to set the current thread. Edges that go from the application to the framework represent edges with a source vertex from the methods found in the APK file and a target vertex of a method belonging to the Android or Java framework. An example would be a method call from an instantiated `int` object calling the Java `toString()` method. Edges that go from the framework to the application represent edges with a source vertex of a method belonging to the Android or Java framework and a target vertex from the methods found in the APK file. The edges from framework to application represent callbacks where the framework calls a method in the application. An example would be when an application sets the `OnClickListener`, which is an event triggered method, to a method within the application. When the event is triggered, the framework calls the method set in the listener.

The next step is adding sub-categories to refine our results. To explain the reasoning behind our sub-categorization, we first look into the package, class, method hierarchy. Figure 4·5 is an example of possible interactions within a package. A package may contain multiple classes, in this example Class1 and Class2. In each class, there can be multiple methods. Each method can interact with methods within its

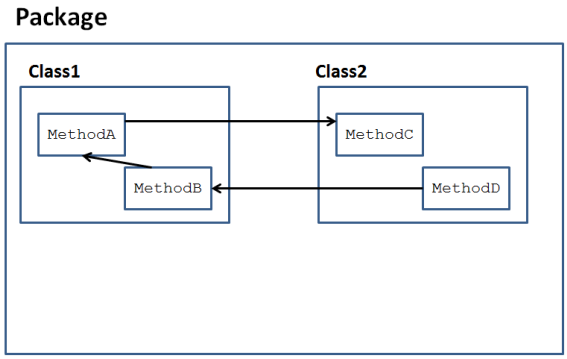

Figure 4·5: Hierarchy of an Application Package

own class and with methods in different classes.

Figure 4·6 shows an overview of how an application with multiple packages interacts with the framework. Based on these interactions, we decided on sub-categories. For the framework, we split the related categories into Java framework and Android framework. For the application we split them into edges that remain in the same class and edges who's target class differs from the source class. The class split was done to be able to identify different method calls within the application. Any edge that does not belong to the list of categories is added to an uncategorized list. Ultimately, the goal is to have no edges added to the uncategorized list.

To eliminate the uncategorized edges, we iteratively added new sub-categories. The first sub-category we added was to the category of within an application. Applications may use multiple packages or package names (see Figure 4·6), so we added a sub-category of package to package. Packages may be imported into an application as an entire project, as bytecode or they are dynamically loaded at runtime. For example, advertising packages are imported into an application as a third party package.

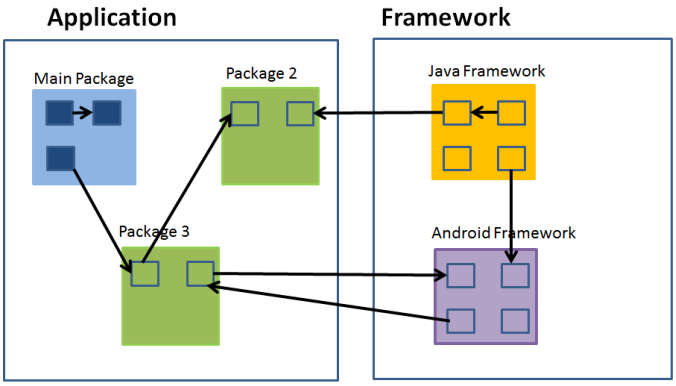

Figure 4·6: Interaction between Application and Framework

During a separate iteration, we decided to distinguish Google's advertising packages from other external packages as it is the most popular advertising library Viennot et al. (2014). By the end of the iterative process, the only values we found in the uncategorized list were from imported packages interacting with the framework. We added the two sub-categories package to framework and framework to package. This resulted in no edges in the uncategorized list.

We tagged the edges in the dynamic call graph with the main categories (see Chapter 4.2). The tool traverses the list of missed edges and places them in the correct sub-category based on the package names of the source vertex method and target vertex method. For example, an edge who's source method belongs to package `Package.example`, where `Package.example` is the application's main package name, and target method belongs to `java.lang` will belong to the sub-category application to Java framework. Each sub-category has its own Java list. The application's package name is used in this step to identify the methods/classes that belong to this application and ones that are imported. Once all edges have been categorized, the

lists of categories and their edges are written to a JSON file. This file is the final output of `EdgeAnalyzer`. It contains the statistics of each category with the missing edges grouped by category per application.

To collect statistics on multiple applications, we implemented a JSON parser independent of `EdgeAnalyzer`. We named the statistics collector `statTool`. `statTool` takes as its input the location of a directory and then traverses the directory looking for the JSON files created by `EdgeAnalyzer`. Each JSON file `statTool` finds gets parsed to find the number of edges missed per category. `statTool` also keeps track of the number of applications that miss particular categories. Once all the JSON files in the directory have been parsed, it outputs the statistics of the overall analysis.

Chapter 5

Evaluation

To evaluate this project we answer the following questions:

1. *How unsound are static call graphs?*
2. *How widespread are missed edges in real world applications?*
3. *What does categorization of missed edges reveal?*
4. *What are the implications of missed edges?*

1 *How unsound are static call graphs?*

To answer this question we developed four Android applications; one that we expected FlowDroid to accurately create a call graph for and three we expected to have missed edges. Our expectation was based on the notion that it may be difficult for FlowDroid to correctly handle the three applications. For each of these applications, we first run our modified version of FlowDroid to export the call graph into a dot file.

The first application `testSMS` takes user input in the form of text and sends the text via SMS when the button on the main screen is clicked. As a straightforward application we expected FlowDroid to accurately create its call graph. The application was on an emulator running our modified version of AOSP. We started the application, typed random text and clicked on the button on the main screen. The log file was then pulled(copied) from the emulator and used with the dot file as input to our

```
<Button
   android:layout_height="wrap_content"
   android:layout_width="wrap_content"
   android:id="@+id/button1"
   android:text="@string/send_sms"
   android:onClick="sendSMSMessage"
/>
```

Figure 5·1: `OnClickListener` in the XML

```
sendBtn.setOnClickListener(new View.OnClickListener() {
   public void onClick(View view) {
      sendSMSMessage();
   }
});
```

Figure 5·2: `OnClickListener` in the Java Code

tool. We were surprised to see that the methods called by the `onClickListener` were in the missed edges list. To confirm that the edges were missed statically, we manually checked the call graph created by FlowDroid. `testSMS` sets the `OnClickListener` in the XML file(see Figure 5·1). Another way to set the `OnClickListener` is using the Java code instead of the XML(see Figure 5·2). We modified the application and removed the `onClick` from the XML and added it to the `OnCreate` method in the Java code. We ran the experiment again and the result was that all edges were present as originally expected. This indicates that a listener set in the XML of an application will be missed but setting the listener in the Java code will result in a sounder result. A sounder result would be one that misses less edges, in this case the callback from the event `onClick`.

The second application `Inception` requires no user interaction. On startup, a method `First` is called. `First` calls method `secondCall` that throws an exception and in `secondCall`'s catch clause the method `thirdCall` is called (see Figure5·3

```
public void secondCall()
{
   try
   {
      int x=0;
      int y=5/x;
   }
   catch(Exception e)
   {
      thirdCall();
   }
}
```

Figure 5·3: Code from the second app, divide by zero exception

for the code). We ran this application through the same exercise as `testSMS`. Our expectation for this application was that the edge from `secondCall` to `thirdCall` would be missing. The expectation was based on the assumption that exception handling in static analysis is hard Arzt et al. (2014). The result from `EdgeAnalyzer` was that all edges were present.

In the third application, `ToastMaster`, we created a class `Toaster` that extends Android's `Toast` class. The `Toast` class allows developers to display short messages to the user over the application. The message is displayed when the `show()` method is called. We extended `show()` to send an SMS to our own number via a method named `surprise` (see Figure 5·4). In the main application, when the button on the main screen is clicked, an instance of `Toaster` is created and the `show()` method is called. To avoid the missed edges due to `onClickListener` in the XML, we add the code in Figure 5·2. We incorrectly assumed that FlowDroid's call graph would not contain the methods that send the SMS. The assumption was based on the idea that reasoning about extended classes is hard. We ran this application through FlowDroid and on our emulator we clicked on the button on the main screen. The call graphs

```
public class Toaster extends Toast
{
   private Context cntxt;
   @Override
   public void show()
   {
      surprise();
   }
}
```

Figure 5·4: Code from the third app, extension of Toast class

were used as input to our tool and the result was that the edges were present in the call graph.

For the fourth application, `Exceptional`, instead of extending a class from the Android framework, we extend one from Java's framework. We created a class `Exceptional` that extends Java's `Exception` class. `Exception` has a method `getMessage()` that retrieves the exception's detail message. We overwrite `getMessage()` to retrieve the phone's unique identifier number(IMEI) and send it by SMS to our number. In the main application, an `Exceptional` is thrown and the `getMessage()` is called (see Figure 5·5 for code). We again incorrectly assumed that FlowDroid's call graph would not contain the methods that get the IMEI number and send the SMS. The assumption we made was based on the fact that reasoning about exceptions and extended classes is hard. We ran this application through the same exercise as the previous three applications. The result was that the edges were present in the call graph.

The missed edge in the first application indicates a problem in parsing the layout files. FlowDroid parses the file and so should be able to identify that there is a call back registered for the `onClick` method.

```
public class Exceptional extends Exception
{
   TelephonyManager tel;
   public Exceptional()
   {
      super();
   }
   public String getMessage(String t)
   {
      SmsManager smsManager = SmsManager.getDefault;
      smsManager.sendTextMessage("6171234567",null,t,null,null);
      return "hello world!";
   }

}
```

Figure 5·5: Code from the fourth app, extension of Exception class

The result of the first application indicates that based on our definition of soundness, that all edges in the application must be contained in the call graph, the call graph created by FlowDroid is not sound. Based on the four applications, we can claim that call graphs are unsound 25% of the time. However, a pool of four simple applications does not indicate how unsound call graphs are in real world applications.

2 *How widespread are missed edges in real world applications?* We used a pool of 92 real world Android applications to test our tool. We downloaded these applications from the PlayDrone repository Viennot et al. (2014).Playdrone crawls the Google Play store to download applications and categorize them. The applications we chose were from the "Social" category. 19 of the 92 applications failed execution due to lack of heap memory despite a size of 4GB. Our modified version of FlowDroid was run on each of these applications to extract the call graph into a dot file. We used a fresh emulator instance for each application. Android's Monkey tool was run to simulate user interaction on the emulator. Monkey is an application exerciser that randomly

generates key strokes such as click or swipe.

For each of the applications, the application was installed on the running emulator and Android's Monkey program was run with 200 random key strokes. A key stroke is an action that a user can make, for example a finger touching the screen at a certain position. Since the key strokes are randomly selected, we wanted to ensure maximum operations while simultaneously not overwhelming the application. We tested different values for the number of key strokes and 200 was empirically chosen as it gave us adequate coverage and simultaneously did not produce a log file that was unreasonable large in size. At the end of the simulated interaction, the log file containing the logged function calls was extracted from the emulator. An application's package name is written into the `AndroidManifest.xml` file. The package name is used to identify the main package of the application for the categorization of edges. We retrieved the application's package name by using the Android Asset Packaging Tool(aapt) on the APK file and then run our tool with the extracted log file, dot file and the package name.

Using our automated statistics collector, we found that 62 applications have edges missed in their static call graph (see Figure 5.1). This significantly changes the claim of soundness of call graphs. Based on these results, the call graph is unsound 84.9% of the time. This is not an indicator of vulnerabilities or malicious activity missed but invalidates assumptions based on the soundness of call graphs.

3 *What does categorization of missed edges reveal?*

As explained in Chapter 4.4, we categorized all the edges that were missed. To verify that edges are in the correct categories, we inspected 10 of the JSON files. As expected, all edges were correctly categorized. To be able to quantify the missed edges and their categories, we ran our statistics collecting tool, statTool. The results of the collection are in Table 5.2.

Table 5.1: App Statistics that show the number of applications with specific categories of edges missed

From package to framework	61
From framework to package	52
In different packages	51
Within the main package	27
From framework to app	43
From app to framework	42
From ad library to framework	4
From ad framework to library	2
Within the ad library	2
Uncategorized edges	0
Total number of apps	73

Table 5.2 is ordered based on the most number of missed edges per category. The categories framework to package and package to framework are related to the packages that are not the main package. The numbers reveal that the majority of missed edges are related to packages that are not the package declared in the `AndroidManifest.xml` file. This indicates that the call graph does not accurately represent the interactions with third party libraries. It is also evident that call graphs unsoundness is not limited to a specific set of categories. This indicates that there is more than one cause for the edges to be missed.

Table 5.2: The numbers of missed edges in the categories

Category of Missed Edges	Total
Package To Framework	9,907
Inter-package	6,323
App To Android Framework	1,067
Framework To Package	803
Class To Class	483
Android Framework To App	430
App To Java Framework	324
Ad To Framework	108
Same Class	106
Ad To Ad	36
Framework To Ad	10
Java Framework To App	1
Other	0
Total	19,653

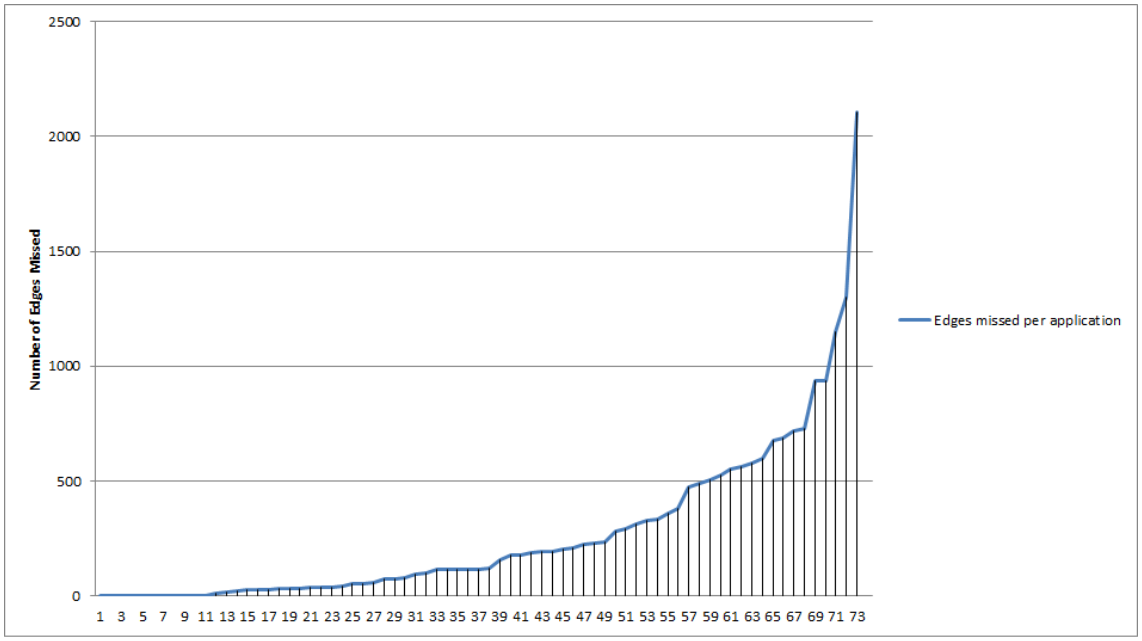

Figure 5·6: Distribution of missed edges per app

Figure 5·6 is a graphical representation of the number of missed edges per application. The vertical axis represents number of edges missed by the static call graph. The graph shows us that 59 of the 73 applications have less than 500 edges missing and that one application has more than 2000 edges missing.

4 *What are the implications of missed edges?*

To determine the cause of the missed edges, we manually checked a few of the results to verify our expectations of finding missed edges that could potentially alter the results of an analysis. Categories that we identified as interesting were inter-package edges and intra main package edges. We looked into two applications with intra main package edges, that is intra-class and inter-class. One of the two applications missed edges that were related to the `onClick` issue we mentioned earlier. Thus our take-away observation is that the parsing of the layout files should be improved to handle the case. The second application's missed edge was unrelated to a callback method. The initial verification was done by looking at the file with the log of traced

```
ar.com.indiesoftware.peoplesay.android.PeopleSayApplication:
boolean hasNetworkActivity()
->
ar.com.indiesoftware.peoplesay.android.services.DataService:
int getConnections()
```

Figure 5·7: Missed edge from an application

edges. The edge that was missed is shown in Figure 5·7. The edge missed has a start vertex of method `hasNetworkActivity()` from class `PeopleSayApplication` and end vertex of method `getConnections()` from class `DataService`. On manual inspection, we verified that the vertex `hasNetworkActivity()` belongs to the static call graph. To verify the existence of the methods in the application code, we used apktool to disassemble the application. Once verified, we used the Dex2jar tool to decompile the application and find an explanation for the missed edges. Figure 5·8 shows that the method `getConnections()` is called based on the condition that the object `service` exists. An assumption for the missed edge could be that the object `service` is never set and therefore the analysis correctly assumes that the method call `getConnections()` will never be called. Our dynamic call graph shows that this assumption is incorrect. Based on our results, the object exists and the method is called. This indicates that analysis that looks into data-flow by propagating taint (such as FlowDroid) will inaccurately miss propagating through this edge.

We identified 27 applications with edges within the application itself, using the same package, that are missed statically but found dynamically(see Table 5.1). The two examples above fit into this category of applications.

We picked three application with missed edges from a package other than the package declared in the `AndroidManifest.xml` file. We disassembled the applications using apktool to see whether the code for the packages were dynamically loaded. A package that is dynamically loaded will be included in the application during runtime

```
public boolean hasNetworkActivity()
{
   boolean bool2 = false;
   boolean bool1 = bool2;
   if (this.service != null)
   {
      bool1 = bool2;
      if (this.service.getConnections() != 0) {
         bool1 = true;
      }
   }
   return bool1;
}
```

Figure 5·8: The code

and therefore its edges cannot be found by static analysis tools. All three applications statically linked the third party packages. In other words, we could see the package code in the disassembled application. This means it was not dynamically loaded and should have been part of the static call graph. An issue with third party libraries is the fact that the library shares the application's permissions. Knowing that edges from statically linked third party libraries could be missed indicates that there needs to be a more thorough examination of static code with reference to these libraries. Our observation is that the handling of statically linked third party libraries needs to be improved to correctly assess an application statically.

We state that any edge observed dynamically but not present in the static call graph is a witness for unsoundness. The set of edges we observed dynamically but were not present in the static call graph is a lower bound of edges. The analysis shows that the call graphs were unsound 84.9% of the time. The categorization of the missed edges highlighted three main issues. The first issue is the analysis of third party libraries statically linked to the application. The second issue is the incorrect

handling of the `onClick` callback from the layout files. The third issue is the missed edge due to the incorrect assumption of a condition never being met. All three issues should be handled correctly to obtain a sounder call graph. An unsound call graph will render assumptions based on its soundness invalid. Therefore the conclusions derived from a static analysis tool will also be unsound.

Chapter 6

Related Work

The following is a discussion of related scientific literature in the areas of program analysis and analysis tools. We present various tools in static and dynamic analysis of Android applications. Our work has benefited from related work and is also complimentary to it. Our focus is on finding and highlighting gaps in static analysis tools for further development efforts to focus on.

Soot Vallée-Rai et al. (1999) is a tool used by researchers and developers to develop static analysis tools for Java programs. A key feature of the Soot framework is its support for implementing intra-procedural data-flow analyses. This includes the generation of call graphs using Soot's construction algorithms through which call graph information is provided. Soot uses an over-approximation of the set of calls that may occur at runtime to compute, for each call, all possible targets of that call site. Soot generates call graphs of Java code. Since Android is programmed in Java, it can, and has been used to generate call graphs for Android applications. These call graphs can be used as input to our tool to identify areas of unsoundness.

FlowDroid Arzt et al. (2014) is a precise context, flow, field, object-sensitive and lifecycle-aware static analysis tool. FlowDroid extends soot to create the call graphs. FlowDroid handles the complexity of not having a standard main method in an Android application by creating a dummy main method based on the components it finds in the application. It assumes that any of these components may be the starting point of the application. It attaches callbacks to these components. To find the callbacks

registered to each component, FlowDroid creates a call graph for each one. Then each call graph is scanned iteratively to find and add callbacks to the graph. This recursive step stops when no more callbacks are added to any of these call graphs. FlowDroid then performs taint propagation on these call graphs to find sensitive flows. We have shown that FlowDroid does not handle certain instances of callback methods. For example, if a developer decides to set the onClick listener in one of the XML files of the application, this callback and any edges with this method as a source will be missing from the call graph.

Amandroid Wei et al. (2014) is an inter-component data flow analysis framework. The developers shy away from the traditional extension of well known tools such as soot Octeau et al. (2010) and write a decompiler. Amandroid produces a highly precise flow and context sensitive inter-procedural control flow graph (ICFG) of an application. Amandroid includes Inter-component communication edges which our tool also finds. The developers create and use a inter-component data flow graph and a data dependency graph to find sensitive flows. The difference between Amandroid and `EdgeAnalyzer` is that they focus on a specific flow issue, inter-component data flow, where we are looking at the call graph that can lead to different types of missed flows.

EdgeMiner Cao et al. (2015) is a tool that finds implicit control flows through the Android framework. The authors extend FlowDroid as an example of how their tool can be integrated into a static analysis tool. The way flows are calculated in FlowDroid is by using taint propagation on the call graph. It can be extrapolated that without the edge in the call graph, any flow that passes through that edge will be missed. EdgeMiner creates a call graph of the framework instead of the application to perform inter-procedural data flow analysis. The developers note that their procedure does not guarantee precision when it comes to reflection and native code. Due to the

dynamic analysis done in our tool, we can capture these edges under the condition that the execution path taken during the run calls the methods.

IccTA Li et al. (2015) is an open-source static taint analysis tool focusing on communications between an Android application's components to detect sensitive personal data leaks. The main premise of IccTA is that analyzing components separately (Intra-components) is not enough since much of the communication and information exchange is happening between components. Thus, IccTA implements an inter-components communication analysis. IccTA transforms an application's Dalvik bytecode into Jimple representation, then extracts the inter-components communication links and other related data, and connects the components. To handle lifecycle and callback methods, IccTA generates a "dummymain" method for each component to use in the analysis. It then uses a modified version of FlowDroid to build a control flow graph of the application and performs the static taint analysis using stored context information such as the values of intents. Declared limitations of IccTA are failure in resolving reflective calls if their arguments are implicit, unawareness of multi-threading, inability to handle native calls and complicated string operations, and excessive memory consumption. Implicit arguments are defined at runtime and therefore it is difficult to reason about them statically. Since IccTA uses FlowDroid, any unsoundness in FlowDroid can be propagated. `EdgeAnalyzer` has already identified three areas where FlowDroid's call graph is unsound. This unsoundness will render any assumptions IccTA make based on FlowDroid's soundness invalid.

Chapter 7

Conclusions

In this thesis, we designed and implemented a tool, `EdgeAnalyzer`, to find and categorize edges missed in the call graphs of Android static analysis tools. We used dynamic analysis to find edges missed in the creation of static call graphs. We identified unsoundness in call graphs of a state of the art static analysis tool.

Our approach was to modify a static analysis tool to output static call graphs, and modify AOSP to output dynamic call graphs. We used our tool to map the edges of the two call graphs and find missing edges from the static call graph and then categorize them.

We modified FlowDroid, a state of the art precise context, flow, field, object-sensitive, and lifecycle-aware static analysis tool for Android applications Arzt et al. (2014), to print its call graph to a dot file. We used our modified AOSP version to print dynamic call graphs. We used 92 real world Android applications and ran them through the modified versions of FlowDroid and AOSP to generate their static and dynamic call graphs. Using the tool we developed, we successfully showed that there are areas of unsoundness in the static call graph created in FlowDroid. We discussed the implications of the results we found from real world applications. We created a tool that can be used to identify areas of weakness in the creation of call graphs for Android application.

References

Android (2016). `http://developer.android.com/`. [Online; accessed 21-Mar-2016].

Arzt, S., Rasthofer, S., Fritz, C., Bodden, E., Bartel, A., Klein, J., Traon, Y. L., Octeau, D., and Mcdaniel, P. (2014). FlowDroid : Precise Context , Flow , Field , Object-sensitive and Lifecycle-aware Taint Analysis for Android Apps. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 259–269.

Backes, M., Bugiel, S., Hammer, C., Schranz, O., and von Styp-Rekowsky, P. (2015). Boxify: Full-fledged app sandboxing for stock android. In *24th USENIX Security Symposium (USENIX Security 15)*, pages 691–706, Washington, D.C. USENIX Association.

Barros, P., Just, R., Millstein, S., Vines, P., Dietl, W., d'Amorim, M., and Ernst, M. D. (2015). Static analysis of implicit control flow: Resolving Java reflection and Android intents. In *Proceedings of the International Conference on Automated Software Engineering (ASE)*, pages 669–679, Lincoln, NE, USA.

Cao, Y., Fratantonio, Y., Bianchi, A., and Egele, M. (2015). EdgeMiner : Automatically Detecting Implicit Control Flow Transitions through the Android Framework. (February). doi: 10.14722/ndss.2015.23140.

Chess, B. and McGraw, G. (2004). Static analysis for security. *IEEE Security & Privacy*, 2(6):76–79.

Chin, E., Felt, A. P., Greenwood, K., and Wagner, D. (2011). Analyzing inter-application communication in android. In *Proceedings of the 9th International Conference on Mobile Systems, Applications, and Services*, MobiSys '11, pages 239–252, New York, NY, USA. ACM. doi: 10.1145/1999995.2000018.

Chugunov, G., Fredlund, L. A., and Gurov, D. (2002). Model checking of multi-applet javacard applications. In *Proceedings of the 5th Conference on Smart Card Research and Advanced Application Conference - Volume 5*, CARDIS'02, pages 10–10, Berkeley, CA, USA. USENIX Association.

EazyTutz (2015). Android Architecture. `http://www.eazytutz.com/android/android-architecture/`. [Online; accessed 24-Mar-2016].

Enck, W., Gilbert, P., Han, S., Tendulkar, V., Chun, B.-G., Cox, L. P., Jung, J., McDaniel, P., and Sheth, A. N. (2014). Taintdroid: An information-flow tracking system for realtime privacy monitoring on smartphones. *ACM Transactions on Computer Systems*, 32(2):5:1–5:29. doi: 10.1145/2619091.

Evans, B. (2015). The (lack of) app store metrics. `http://ben-evans.com/benedictevans/2015/6/13/the-lack-of-app-store-metrics`. [Online; accessed 21-Mar-2016].

GraphViz (2016). `http://www.graphviz.org/content/dot-language`. [Online; accessed 21-Mar-2016].

Klieber, W., Flynn, L., Bhosale, A., Jia, L., and Bauer, L. (2014). Android taint flow analysis for app sets. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on the State of the Art in Java Program Analysis*, SOAP '14, pages 1–6, New York, NY, USA. ACM. doi: 10.1145/2614628.2614633.

Li, L., Bartel, A., Bissyand, T. F., Klein, J., Traon, Y. L., Arzt, S., Rasthofer, S., Bodden, E., Octeau, D., and McDaniel, P. (2015). Iccta: Detecting inter-component privacy leaks in android apps. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering (ICSE)*, volume 1, pages 280–291. doi: 10.1109/ICSE.2015.48.

Li, L., Bartel, A., Klein, J., Le Traon, Y., Arzt, S., Rasthofer, S., Bodden, E., Octeau, D., and McDaniel, P. (2014). I know what leaked in your pocket: uncovering privacy leaks on Android Apps with Static Taint Analysis. *ArXiv e-prints*.

Livshits, B., Sridharan, M., Smaragdakis, Y., Lhotk, O., Amaral, J. N., Chang, B.-Y. E., Guyer, S. Z., Khedker, U. P., Mller, A., and Vardoulakis, D. (2015). In defense of soundiness: A manifesto. *Communications of the ACM*, 56(1).

McClurg, J., Friedman, J., and Ng, W. (2013). Android privacy leak detection via dynamic taint analysis. *Department of EECS, Northwestern University*. `http://www.jrmcclurg.com/papers/internet_security_final_report.pdf`.

Octeau, D., Enck, W., and McDaniel, P. (2010). The ded Decompiler. *Network and Security Research Center, Department of Computer Science and Engineering, Pennsylvania State University, University Park, PA, USA, Tech. Rep. NAS-TR-0140-2010*, 2010(May).

Octeau, D., McDaniel, P., Jha, S., Bartel, A., Bodden, E., Klein, J., and Le Traon, Y. (2013). Effective inter-component communication mapping in android with epicc: An essential step towards holistic security analysis. In *Proceedings of the 22Nd USENIX Conference on Security*, SEC'13, pages 543–558, Berkeley, CA, USA. USENIX Association.

Review, M. (2015). The Truth About Smartphone Apps That Secretly Connect to User Tracking and Ad Sites. `https://www.technologyreview.com/s/537186/the-truth-about-smartphone-apps-that-secretly-connect-to-user-tracking-and-ad-sites/`. [Online; accessed 21-Mar-2016].

Rice, H. G. (1953). Classes of recursively enumerable sets and their decision problems. *Transactions of the American Mathematical Society*, 74(2):358–366.

Sharma, R. (2011). Developing for Android An Introduction. . `http://www.cprogramming.com/android/android_getting_started.html`. [Online; accessed 24-Mar-2016].

Spreitzenbarth, M., Schreck, T., Echtler, F., Arp, D., and Hoffmann, J. (2015). Mobile-sandbox: Combining static and dynamic analysis with machine-learning techniques. *International Journal of Information Security*, 14(2):141–153. doi: 10.1007/s10207-014-0250-0.

Vallée-Rai, R., Co, P., Gagnon, E., Hendren, L., Lam, P., and Sundaresan, V. (1999). Soot - a java bytecode optimization framework. In *Proceedings of the 1999 Conference of the Centre for Advanced Studies on Collaborative Research*, CASCON '99, pages 13–. IBM Press.

Viennot, N., Garcia, E., and Nieh, J. (2014). A measurement study of google play. In *The 2014 ACM International Conference on Measurement and Modeling of Computer Systems*, SIGMETRICS '14, pages 221–233, New York, NY, USA. ACM. doi: 10.1145/2591971.2592003.

Wei, F., Roy, S., and Ou, X. (2014). Amandroid: A Precise and General Inter-component Data Flow Analysis Framework for Security Vetting of Android Apps. *Computer and Communications Security (CCS)*, pages 1329–1341. doi: 10.1145/2660267.2660357.

Xie, Y. and Aiken, A. (2005). Context- and path-sensitive memory leak detection. In *Proceedings of the 10th European Software Engineering Conference Held Jointly with 13th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, ESEC/FSE-13, pages 115–125, New York, NY, USA. ACM. doi: 10.1145/1081706.1081728.

CURRICULUM VITAE

Dana Aljawder
E-mail: djawder@bu.edu

Education

- **Boston University** College of Engineering, Boston MA
  Master of Science in Computer Engineering, Specialization: Cyber-security Expected May 2016
- **American University of Sharjah** College of Engineering, Sharjah United Arab Emirates
  Bachelor of Science in Computer Engineering 2011

Thesis

"Identifying Unsoundness in Call Graphs of Static Analysis Tool for Android"

Proposed and developed a novel approach to automatically identify unsoundness of static call graphs by using a dynamic call graph.

Thesis Advisor Manuel Egele

Professional Experience

November 2011 - July 2014 **ADNOC Distribution Abu Dhabi, U.A.E.**

IT Project Analyst

- Developing and managing in house developed Intranet & Internet applications in the areas of Retail, Finance, Human Resources, Information Technology, Strategy and Procurement
  - Developing applications using ASP classic, ASP.net and Oracle PL/SQL
  - Implementing security features for Internet Websites
- Developing and Managing in-house E-Forms with workflows
- Design and implementation of a database for an IT Knowledge Base system
- Managing Microsoft's EPM Solution
- Setup of Internet server
- End user training and support

Summer 2009 **Injazat Datasystems Abu Dhabi, U.A.E.**

Summer Trainee

- Monitored and managed servers
- Monitored and trained in firewalls and DDOS in Security
- Trained in network operations

Related Projects

The following were group projects part of my Master's program

Android Ad Vulnerability

- Exploited vulnerabilities presented by Javascript and shared permissions in Android applications by developing a malicious 3rd party library and linking it to a benign application. Demonstrated data leakage through the application.

Dynamically Detecting Injection on a LAMP Web Application

- Modified the PHP interpreter on a LAMP stack to perform fine-grained taint analysis on user input.

Exploiting Oauth Vulnerabilities

- Implemented automation tools to detect and exploit vulnerabilities in sites that use Oauth authentication.

Propagating a Geo-Targeted Worm

- Simulated the propagation of a geo-targeted worm. Developed a proof-of-concept Android application that propagates using Bluetooth towards a specified direction.

Web Interface for HaaS on MOC

- Integrated and modified the API calls for the Massachusetts Open Cloud(MOC) Hardware as a Service (HaaS) into a web application to improve the end user experience.

Maps Web Application

- Developed a web application that takes an input of over 2.2 million coordinates as reference points, creates an R-Tree to store the data. The interface uses Google maps and responds to two query types: 10 nearest reference points and reference points with an area.

Training Courses

Technical:

- Microsoft Visio L1

- ASP.net Visual studio 2007
- IT Project Management
- Programming in Oracle PL/SQL

Non-Technical:

- Personal Core Competency
- Presentation Skills
- Business Memo and Report Writing Skills
- Negotiation Skills

Programming Skills

C++/C/C#, Java, Javascript/HTML/CSS, PHP, ASP.net, ASP Classic, XML, sql, Oracle PL/SQL, python